

实验二：图像增强参考文档

一、实验目的

学会OpenCV的基本使用方法，利用OpenCV等计算机库对图像进行平滑、滤波等操作，实现图像增强。

二、实验内容

2.1 导入图像滤波相关的依赖包

- 1.OpenCV库：计算机视觉与图像处理
- 2.scikit-image库：图像增强与分析，random_noise 用于给图像添加噪声
- 3.NumPy库：科学计算的基础库
- 4.Matplotlib库：绘图与数据可视化

```
# =====导入依赖库=====
import cv2
from skimage.util import random_noise
import numpy as np
from matplotlib import pyplot as plt
```

2.2 读取原始图像并进行色彩空间转换

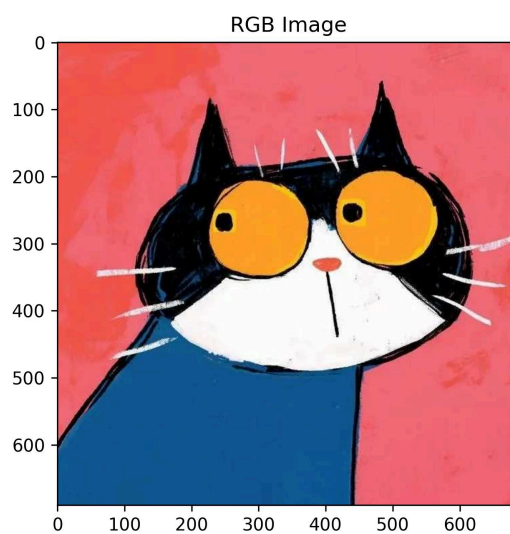
读取计算机本地图像文件，获取并输入【100，100】处像素点的RBG参数并输出,通过下面的输出结果可以看到，【100,100】像素点的RGB为【82,86,241】，表现为偏蓝色，通过图像也可以看到这个像素点属于猫猫的衣服处的深蓝色部位。

```
# =====读取原始图像=====
img = cv2.imread('p1.jpg')
# 获取图像中【100, 100】这个像素的rbg三色
(b, g, r) = img[100, 100]
# 打印这个像素点的rbg参数
print(b, g, r)
# 输出原始图像
plt.imshow(img)
plt.title('Original Image')
plt.savefig('output_images/original_bgr.jpg', dpi=300)
plt.show()
```

- 输出：

82 86 241

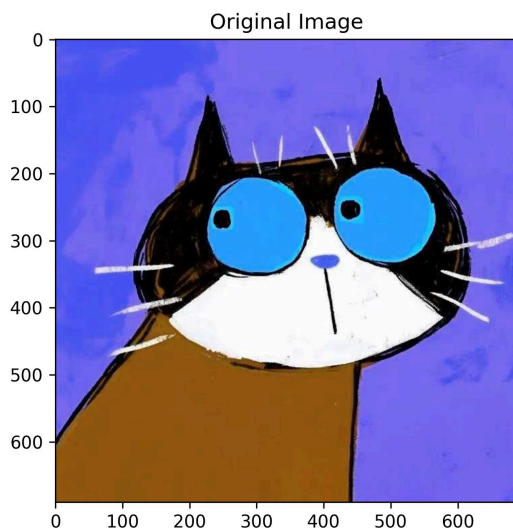
原图：



然后测试一下cv2中颜色空间变换的效果，这里的cv2.cvtColor就是颜色空间转换，cv2.COLOR_BGR2RGB代表的是将原始图像BGR格式转换成RGB格式，蓝色和红色互换，因为把'B'和'R'通道互换了，所以这是一个红蓝的颜色反转

```
# 颜色空间转换，将原始图像BGR格式 转换成 RGB格式，蓝色和红色互换，因为把'B'和
rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb_img)
plt.title('RGB Image')
plt.savefig('output_images/rgb_image.jpg', dpi=300)
plt.show()
```

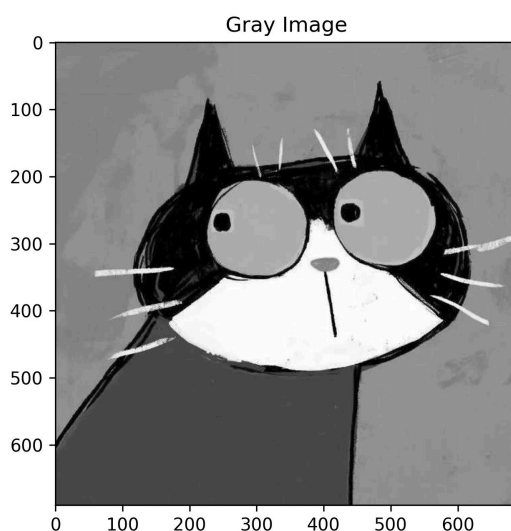
红蓝反转图：



cv2.COLOR_BGR2GRAY是将原始图像的RGB格式转换为灰度图，将三维的RGB通道映射为一维的灰度通道

```
# 将原始图像的RBG格式转换为灰度图
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray_img, cmap='gray')
plt.title('Gray Image')
plt.savefig('output_images/gray_image.jpg', dpi=300)
plt.show()
```

灰度图：



2.3 添加噪声

这里在原始图像的基础上添加噪声，引入了两个API方法，椒盐噪声和高斯噪声，通过对比可以发现，椒盐噪声和高斯噪声的本质不同，椒盐噪声表现为像素会随机替换为白色或者黑色像素（灰度通道），在RGB通道表现为像素变成随机彩色点，而高斯噪声会在每个像素上添加随机偏差，服从高斯分布

```
# =====添加噪声=====

# mode='s&p'代表椒盐噪声，s代表白色，p代表黑色，amount=0.4代表会有40%的像
sp_noise_img = random_noise(rgb_img, mode='s&p', amount=0.4)
# mode='gaussian' 代表高斯噪声，会在每个像素上添加随机偏差，服从高斯分布（均1
gus_noise_img = random_noise(rgb_img, mode='gaussian', mean=0.2, var=0.03)

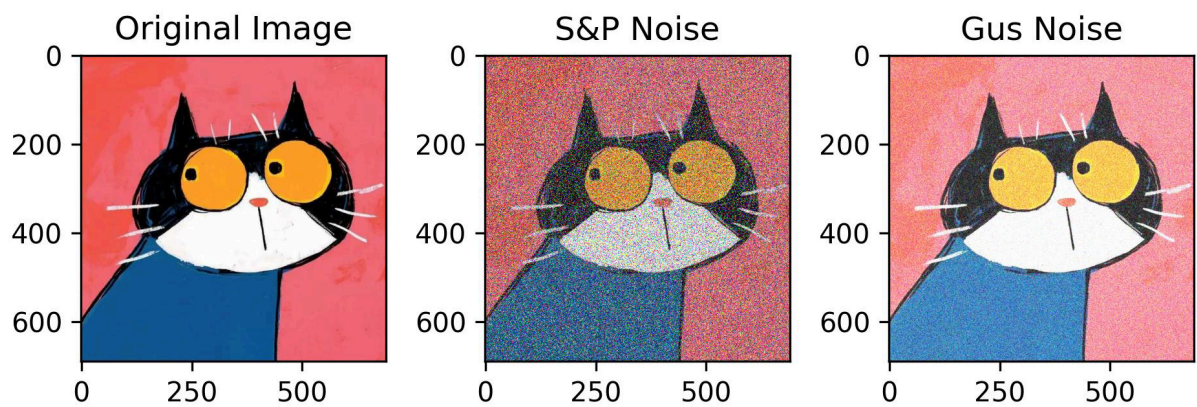
# 原图
plt.subplot(1, 3, 1)
plt.imshow(rgb_img, cmap='gray')
plt.title('Original Image')

# 椒盐噪声
plt.subplot(1, 3, 2)
plt.imshow(sp_noise_img, cmap='gray')
plt.title('S&P Noise')

# 高斯噪声
plt.subplot(1, 3, 3)
plt.imshow(gus_noise_img, cmap='gray')
plt.title('Gus Noise')

plt.tight_layout()
plt.savefig('output_images/noise_comparison.jpg', dpi=300)
plt.show()
```

噪声对比图：



2.4 图像滤波

将图像认为产生噪声后，用OpenCV的三个API滤波方式进行对比，分别对椒盐滤波和高斯滤波使用【均值滤波】，【中值滤波】，【高斯滤波】，对比每个最适合的滤波方式。

```
# =====图像滤波=====

# 均值滤波
mean_sp = cv2.blur(sp_noise_img, (5, 5))
mean_gus = cv2.blur(gus_noise_img, (5, 5))

# 中值滤波（需要 uint8）
mid_sp = cv2.medianBlur((sp_noise_img*255).astype(np.uint8), 5)
mid_gus = cv2.medianBlur((gus_noise_img*255).astype(np.uint8), 5)

# 高斯滤波
gauss_sp = cv2.GaussianBlur((sp_noise_img*255).astype(np.uint8), (5, 5), 0)
gauss_gus = cv2.GaussianBlur((gus_noise_img*255).astype(np.uint8), (5, 5), 0)

# 图像显示
plt.figure(figsize=(13, 9))

# 第1行：椒盐噪声
plt.subplot(2, 3, 1)
plt.imshow(mean_sp)
plt.title("S&P noise with Mean Filter")

plt.subplot(2, 3, 2)
plt.imshow(mid_sp)
plt.title("S&P Noise with Median Filter")

plt.subplot(2, 3, 3)
plt.imshow(gauss_sp)
plt.title("S&P Noise with Gaussian Filter")

# 第2行：高斯噪声
plt.subplot(2, 3, 4)
plt.imshow(mean_gus)
```

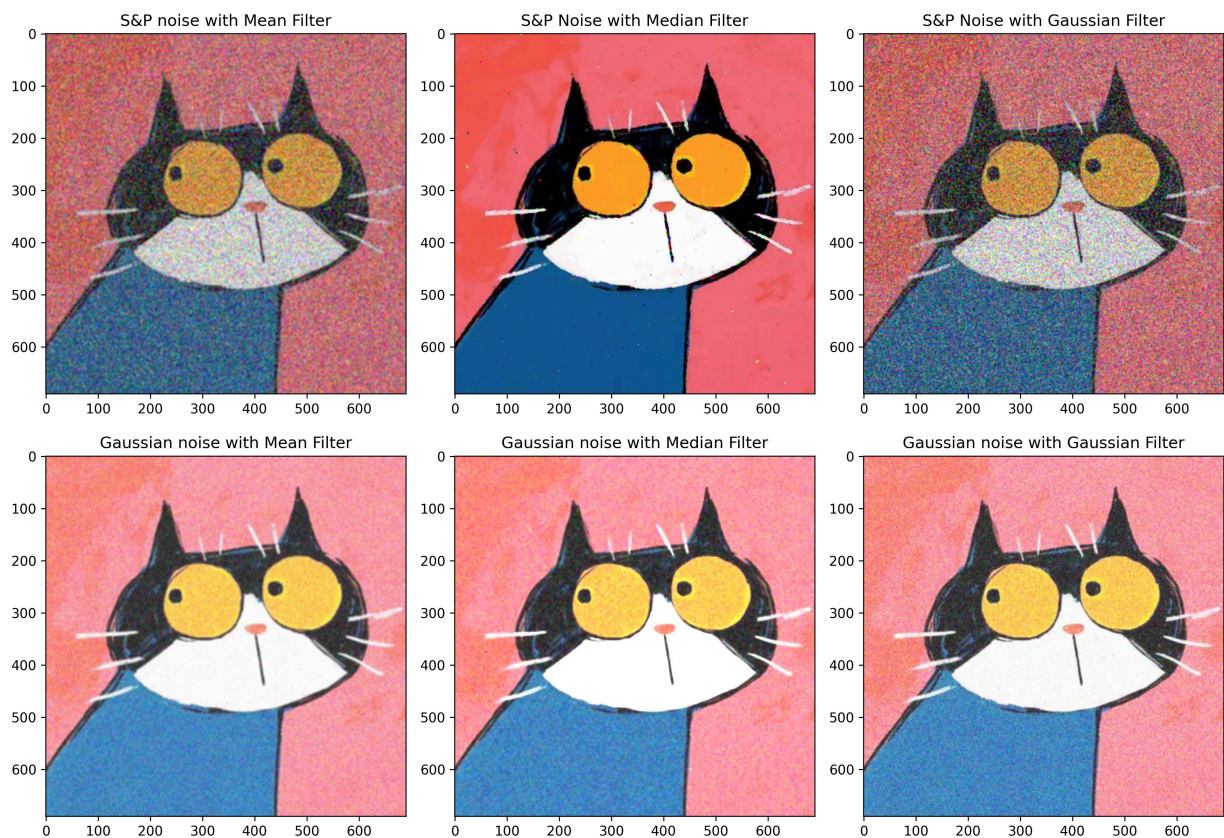
```
plt.title("Gaussian noise with Mean Filter")

plt.subplot(2, 3, 5)
plt.imshow(mid_gus)
plt.title("Gaussian noise with Median Filter")

plt.subplot(2, 3, 6)
plt.imshow(gauss_gus)
plt.title("Gaussian noise with Gaussian Filter")

plt.tight_layout()
plt.savefig('result/filter_results_2x3.jpg', dpi=300)
plt.show()
```

图像滤波对比图：



2.5 手动实现一个滤波方式（中值滤波）

```
# =====手动实现中值滤波=====

def manual_median_filter_color(image, kernel_size=5):
    """
    手动实现彩色图像的中值滤波
    :param image: 输入彩色图像，numpy数组，shape=(H, W, 3)
    :param kernel_size: 滤波窗口大小
    :return: 中值滤波后的彩色图像
    """

    pad = kernel_size // 2
    # 对每个通道单独处理
    filtered_img = np.zeros_like(image)

    for c in range(3): # 遍历通道 R,G,B
        channel = image[:, :, c]
        padded_channel = np.pad(channel, pad_width=pad, mode='edge')
        for i in range(channel.shape[0]):
            for j in range(channel.shape[1]):
                region = padded_channel[i:i + kernel_size, j:j + kernel_size]
                filtered_img[i, j, c] = np.median(region)

    return filtered_img

# 对之前添加椒盐噪声的图像进行手动中值滤波
manual_mid = manual_median_filter_color((sp_noise_img * 255).astype(np.uint8), k

# 显示对比
plt.figure(figsize=(8, 4))

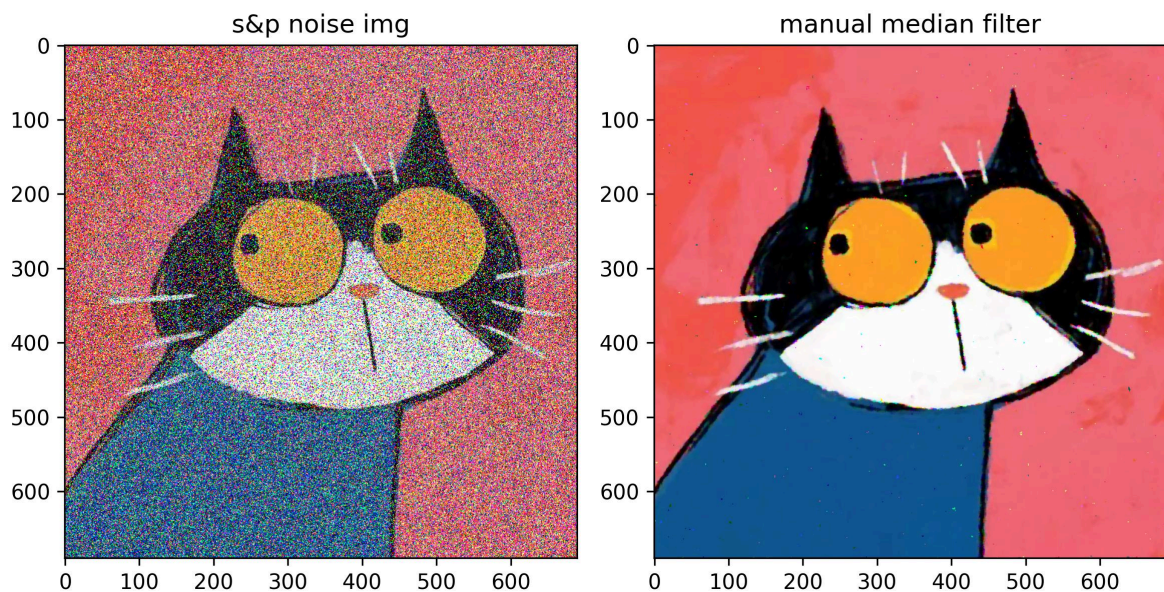
plt.subplot(1, 2, 1)
```

```
plt.imshow(sp_noise_img, cmap='gray')
plt.title("s&p noise img")

plt.subplot(1, 2, 2)
plt.imshow(manual_mid, cmap='gray')
plt.title("manual median filter")

plt.tight_layout()
plt.savefig('result/manual_median.jpg', dpi=300)
plt.show()
```

手动构建中值滤波效果图：



三、实验结果与分析

1.原始图像与颜色空间转换

- 通过 OpenCV 读取图像并显示，可以清楚看到原始彩色图像的细节。

- 将 BGR 格式转换为 RGB 格式后，颜色显示更符合人眼认知，红、绿、蓝通道正确对应。
- 灰度图显示则突出图像的亮度信息，有助于后续图像处理分析。

2.噪声添加效果

- 椒盐噪声 (s&p noise)：在图像中随机出现黑白点，使图像部分区域明显破坏。
- 高斯噪声 (Gaussian noise)：整个图像亮度轻微抖动，更加均匀，整体细节略模糊。
- 对比实验显示，椒盐噪声的局部破坏更明显，高斯噪声则影响全局视觉效果。

3.滤波去噪效果

- 均值滤波 (Mean Filter)：

对高斯噪声的去除效果较好，能够平滑整幅图像，但对椒盐噪声中的尖锐黑白点去除不彻底，且容易造成边缘模糊。

- 中值滤波 (cv2.medianBlur)：

对椒盐噪声的去除效果显著，能有效保留图像边缘和细节；对高斯噪声也有一定的抑制作用，但整体去噪效果略逊于均值滤波。

- 高斯滤波（cv2.GaussianBlur）：

对高斯噪声的去除效果最佳，平滑自然，且保留了一定边缘信息；但对椒盐噪声的处理能力有限，因为其噪声是突变的而非连续分布。

- 手动实现中值滤波（彩色）：

效果与 OpenCV 自带中值滤波基本一致，能有效去除彩色图像中的椒盐噪声，并保持颜色真实和边缘清晰。实验验证了对彩色图像应分别对 R、G、B 三通道独立滤波的合理性。

通过对比显示，手动中值滤波在保持彩色信息、边缘清晰度方面表现良好，说明对彩色图像处理时，需要对每个通道分别滤波。

总体分析

不同类型的噪声适合采用不同的滤波方法进行去噪处理：

- 椒盐噪声（Salt & Pepper Noise）→ 由于其属于突变型噪声，中值滤波（Median Filter）能有效去除孤立的黑白噪点，同时较好地保留图像边缘细节。
- 高斯噪声（Gaussian Noise）→ 属于连续型噪声，适合采用均值滤波（Mean Filter）或高斯滤波（Gaussian Filter）进行平滑处理，能够在抑制噪声的同时保持较自然的视觉效果。

手动实现的彩色中值滤波不仅复现了 OpenCV 中的滤波效果，还加深了对滤波原理的理解。在实现过程中，通过分别对 R、G、B 三个通道独立处理，能够灵活调整滤波核大小和算法逻辑，为后续针对不同噪声类型的自定义滤波与优化提供了良好的基础。

四、实验小结

本实验完成了彩色图像的读取、BGR→RGB 转换以及灰度图显示，并在此基础上为图像添加了椒盐噪声与高斯噪声。

通过分别采用均值滤波、中值滤波以及手动实现的中值滤波对噪声图像进行去噪处理，对比分析了不同滤波方法的效果。

实验结果表明：

- 中值滤波对椒盐噪声的去除效果最为显著，能够有效保留图像边缘和细节；
- 均值滤波更适用于高斯噪声的平滑去除；
- 手动实现的中值滤波在处理彩色图像时同样表现良好，既能有效抑制噪声，又能保持图像的真实色彩与结构信息。

通过本次实验，进一步加深了对图像噪声类型与滤波原理的理解，掌握了手动实现彩色图像中值滤波的基本方法，为后续图像去噪与滤波算法的改进与优化奠定了基础。